

Simulating Your Embedded Project on Your Computer (Part 1)

by Nathan Jones



Mindstorms Engineering, LLC
Embedded Systems Design & Education

Contents

- [Introduction](#)
- [The benefits of simulation](#)
- [First, isolate the madness](#)
- [The simplest option: printf/scanf](#)
 - [Making scanf/getchar non-blocking](#)
 - [A Note About Race Conditions](#)
 - [Making printf prettier](#)
- [What can I do with printf/scanf?](#)
- [Summary](#)
- [Resources](#)

Part 2 of this series can be found [here](#).

Introduction

After reading "[Patterns in the Machine](#)" by John Taylor and lots of stuff on [Embedded Artistry](#), I would say that I had come around to the idea that it was a "good thing" to be able to simulate your embedded system on your computer. I even took what I'd learned in the Embedded Artistry "[Introduction to Build Systems using make](#)" course and [published a makefile](#) that demonstrated how to target a simple "blinky" program to both an STM32 and an x86. On a recent project, though, I wanted to build on this knowledge and I discovered that many of the techniques I had thought I would use to simulate my embedded project were too complicated to be useful and certain techniques that seemed too simple to be useful were actually the perfect blend of both! In this two-part blog series, I'll share with you the arguments in favor of simulation (so, hopefully, you too think its a "good thing") and show you what works (and what doesn't work) to help you to simply, easily, and quickly simulate your embedded project on your computer.

You can see working implementations of many of the techniques discussed in this article and the next (including the two that are demonstrated in the videos below) at [this Github repo](#).

The benefits of simulation

As developers of embedded systems, everything we do is directed at getting a custom piece of hardware and software to work correctly. The code we write isn't *supposed* to run on our laptop or desktop computers, it's supposed to run on a tiny microcontroller; it's supposed to interact with real sensors and send directions to manipulate real motors or screens or BLE modules. Because of this, I think it's not until you've been developing embedded systems for some time that you start to wish there was a way to *not* be so tied to the hardware. Has any of this happened to you?

- You want to start writing code for your embedded project (or maybe your employer is requiring it of you), but you don't know which MCU or sensor is being used, so you feel like you *can't* start writing code.
- Your embedded project requires a complex physical setup, or maybe there's only *one* working setup, so that your ability to actually run your code on the hardware is limited.
- You spend hours debugging an issue in software that turns out to be the result of a loose wire (or other hardware problem).
- The project requirements change partway through development (e.g. the target MCU changes, someone wants to add a web interface, etc) and it becomes a major struggle to refactor the code to meet them.

These are all problems that arise from writing code that is too tightly coupled to the underlying hardware and they all have one simple solution: **find a way to simulate your project on your computer**. If your *actual* project code (minus the truly hardware specific stuff like toggling GPIO pins) can run on your computer, then:

- You can start writing code as soon as you have the project requirements, with no (or little) concern about which MCU or sensor will eventually be selected.
- You can run and test your code on your own, without ever needing to use any physical setup.
- You can be assured that your application code works correctly, isolating faults to the hardware or hardware-interfacing code.
- You can more easily switch out hardware implementations, making it easier to adapt to changing project requirements.

But the benefits don't stop there! Once you have a simulation of your embedded project that runs on your computer:

- You can use programs like Valgrind or other programs that only work on full computers to profile your code or to detect memory management and threading bugs.
- You can use [reverse debugging](#) in GDB, which is like having a full trace of your program's execution while its running!
- Later on in the development of your embedded project, in many cases, you can easily turn your simulator into a dashboard for the code that's running on your microcontroller, giving you the ability to view and change the system as its running.

In short, it's like getting a 10x speed-up on the development of your embedded project! Uri Shaked, created of the [Wokwi](#) simulator, describes exactly how this idea helped him finish a project on an otherwise impossible timeline in his 2024 Embedded Online Conference talk, "[Breaking Good: Why Virtual Hardware Prefers Rough Handling](#)".

The day I learned Simulation is a Superpower

Live Demo time!

I spent half a day recording all the different scenarios, capturing the analog readings.

Then I used the Wokwi Arduino simulation engine (AVR8js) to quickly test all the scenarios whenever I made a change.

It saved the day. Let me show you how!

© EmbeddedOnlineConference.com All rights reserved

EmbeddedOnlineConference.com

The badge.

```
src > examples > the-skullino 2.M X
164 int sample(uint8_t pin, int &minVal) {
165     uint16_t result = 0;
166     uint8_t samplesLeft = 4;
167     for (; samplesLeft && result < 128; samplesLeft--) {
168         result = analogRead(pin);
169     }
170     pinMode(pin, OUTPUT);
171     delayMicroseconds(samplesLeft * 116);
172     minVal = min(result, minVal);
173     result -= minVal;
174     return result;
175 }
176
177 int trackMax(int &maxVal, int value) {
178     maxVal = max(value, maxVal);
179     return value;
180 }
181
182 int lowPass(uint16_t &filter, int value) {
183     filter -= filter >> 5;
184     filter += value;
185     return filter >> 5;
186 }
187
188 bool isAhoy() {
189     uint8_t counter = 0;
190     int lMin = INT16_MIN, lPeak = 0;
191     int rMin = INT16_MAX, rPeak = 0;
192     uint16_t lFilter = 0, rFilter = 0;
193     bool rNext = false;
194     for (int i = 0; i < 2280; i++) {
195         if (digitalRead(TOUCH_PIN)) {
196             return false;
197         }
198     }
199 }
```

Skull Emulator

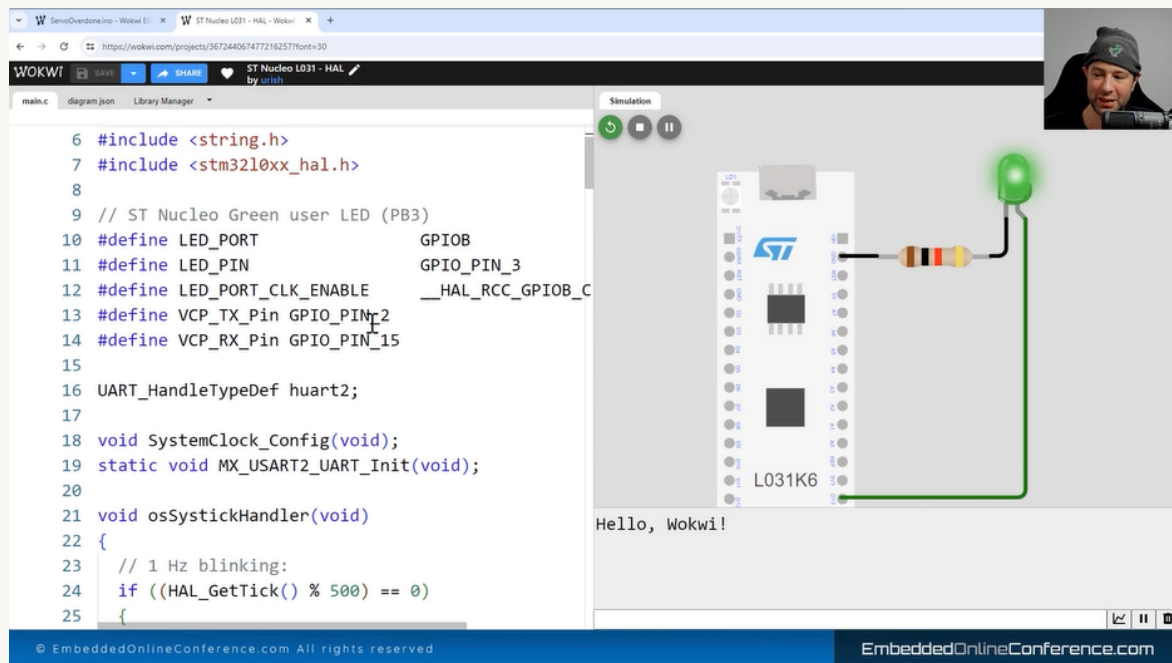
43 tests ran for 4814.50ms
Total simulated time: 25:01 mins

- ✓ E0-Sunlight-Android-Battery-NoAhoy2 🤖💡
- ✓ E2-Arduino-Battery-Adaptive300us-Capped 🤖💡
- ✓ E2-ViaD-Battery-Delay800us 🤖💡
- ✓ E2-AndroidFix-Battery-Adaptive300us2 🤖💡
- ✓ E2-Arduino-Powered 🤖💡
- ✓ E2-Arduino-Powered-Adaptive300us-Daylight 🤖💡
- ✓ E2-Arduino-Powered-Adaptive300usConstantDelay-Daylight2 🤖💡
- ✓ E2-Sunlight-Arduino-Powered 🤖💡
- ✓ E2-Sunlight-Arduino-Powered2 🤖💡
- ✓ E2-Sunlight-Arduino-Powered3 🤖💡
- ✓ E2-Sunlight-Arduino-Powered4 🤖💡
- ✓ E2-Sunlight-Powered-FacingUp 🤖💡
- ✓ E2-Sunlight-Powered-FacingDown 🤖💡
- ✓ E3-Arduino-Battery-A300CD 🤖💡
- ✓ E3-Sunlight-Android-Battery 🤖💡
- ✓ E3-Sunlight-Android-Battery2 🤖💡
- ✓ E3-Sunlight-Arduino-Battery 🤖💡
- ✓ E3-Sunlight-Arduino-Battery2 🤖💡
- ✓ E3-Sunlight-Ariella-Battery 🤖💡
- ✓ E3-Sunlight-Ariella-Battery2 🤖💡

38 🟡 2 ❌ 3 🤖 38 🟡 105 🤖 4

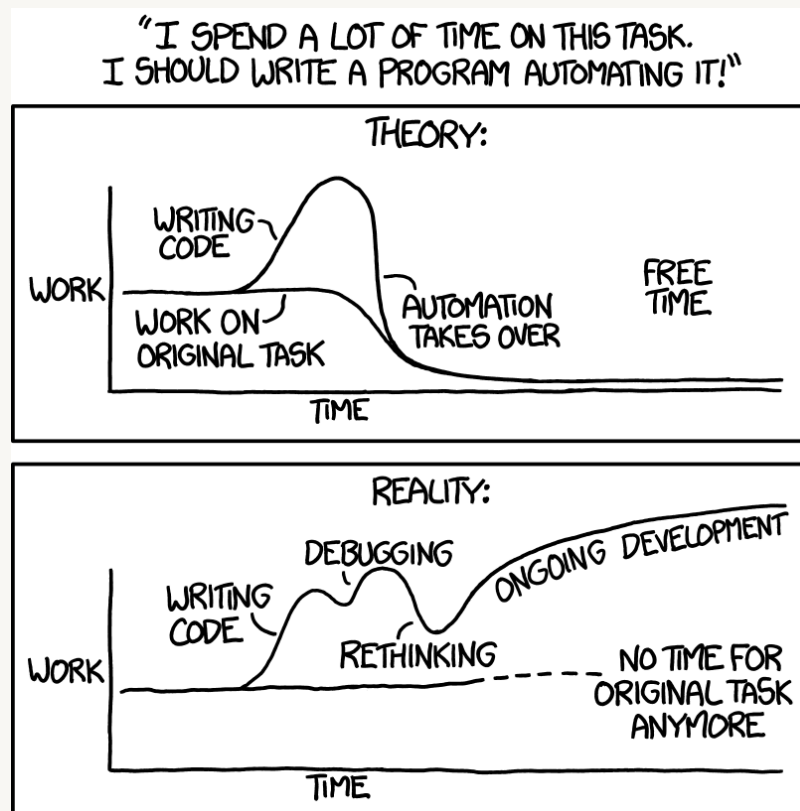
Running the code in a simulation.

(If you've never attended the EOC and don't have access to the video, Uri describes a project he worked on for a conference badge that used LEDs as light sensors. Each test of the firmware took about three minutes to complete and there were 40 different scenarios that needed testing [some in the day and some at night]. And he only had *three days* to write the firmware! Uri describes how he used a Wokwi precursor to run all of his tests at a faster-than-real-life pace so that he could actually take all three days to write his firmware, which he successfully did!)



What the Wokwi simulator looks like now.

There are lots of ways to simulate your embedded system besides Wokwi and I tried many of them out on a recent project. It turns out that some methods are much simpler than others! And simplicity matters, since the whole reason we want a simulator in the first place is to speed up the development of our actual embedded project. Setting up a simulator should only take a *fraction* of the time it will take to complete your project, otherwise we run the risk of spending more time configuring the simulator than we would ever gain by using it.



In this article and the next, I'll show you how to use a number of those techniques (the simpler ones!) to achieve the benefits of simulation that we've identified above.

First, isolate the madness

To be able to simulate (at least part of) your embedded project on your computer, there has to be *some* part of it that doesn't depend on the hardware (unless you're lucky enough to find a simulator that supports your exact processor and your collection of inputs/outputs). If your project currently looks like this (an excerpt from a program demonstrating a simple command-line interface; notice all the references to the STM32 HAL):

```
HAL_UART_Receive_IT(&huart2, cmd, (size_t)1);
while (1)
{
    if( cmd_received )
    {
        // Process the command by comparing it to "on" and "off".
        //
        if( strcmp((char*)cmd, "on") == 0 ) led_is_blinking = true;
        else if( strcmp((char*)cmd, "off") == 0 )
        {
            led_is_blinking = false;
            HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
        }
        else
        {
            uint8_t error_msg[] = "Unknown command\n\r";
            HAL_UART_Transmit(&huart2, error_msg, strlen((char*)error_msg),
HAL_MAX_DELAY);
        }
        // Reset variables to receive another command.
        //
        memset(cmd, 0, strlen((char*)cmd));
        p_current_char = cmd;
        cmd_received = false;
        HAL_UART_Receive_IT(&huart2, cmd, (size_t)1);
    }
    // Non-blocking Blinky. If it's been <interval_ms> since
    // the last time we toggled the LED (<prev_millis>), then
    // toggle the LED.
    //
    if( led_is_blinking )
    {
        if( HAL_GetTick() - prev_millis > interval_ms )
        {
            prev_millis = HAL_GetTick();
            if( HAL_GPIO_ReadPin(LED_GPIO_Port, LED_Pin) == GPIO_PIN_RESET )
            {
                HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
            }
        }
    }
}
```

```

        else
        {
            HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
        }
    }
}
}

```

then you'll need to do some refactoring to make it look like this:

```

initHardware(argc, argv);
while(1)
{
    if( cmd_received )
    {
        // Process the command by comparing it to "on" and "off".
        //
        if( strcmp((char*)cmd, "on") == 0 ) led_is_blinking = true;
        else if( strcmp((char*)cmd, "off") == 0 )
        {
            led_is_blinking = false;
            turnOffLED();
        }
        else
        {
            uint8_t error_msg[] = "Unknown command\n\r";
            send(error_msg);
        }
        // Reset variables to receive another command.
        //
        resetUART();
    }
    // Non-blocking Blinky. If it's been <interval_ms> since
    // the last time we toggled the LED (<prev_millis>), then
    // toggle the LED.
    //
    if( led_is_blinking )
    {
        if( getMillis() - prev_millis > interval_ms )
        {
            prev_millis = getMillis();
            if( ledIsOff() )
            {
                turnOnLED();
            }
            else
            {
                turnOffLED();
            }
        }
    }
}

```

```

    }
}
}
}

```

Notice how the second listing makes *no* assumptions about what system the code is running on nor even *how*, exactly, the LED is turned on or off; as long as the functions shown above behave like we expect them to, the application code is none the wiser. (I like to pass `argc` and `argv` to `initHardware()` in case I want to use them later on to modify the runtime behavior of the desktop version.)

To get to this point, simply pull out all of your hardware-specific code and put it into its own source file (I like to name these after the processor that I intend them to run on, such as "x86.c" or "STM32.c"). In this case, that file might look like this:

```

*****
*           STM32.c           *
*****
#define LED_Pin GPIO_PIN_3
#define LED_GPIO_Port GPIOB
UART_HandleTypeDef huart2;
void initHardware(int argc, char ** argv)
{
    HAL_UART_Receive_IT(&huart2, cmd, (size_t)1);
}
void send(char * msg)
{
    HAL_UART_Transmit(&huart2, msg, strlen((char*)msg), HAL_MAX_DELAY);
}
void resetUART(void)
{
    memset(cmd, 0, strlen((char*)cmd));
    p_current_char = cmd;
    cmd_received = false;
    HAL_UART_Receive_IT(&huart2, cmd, (size_t)1);
}
int getMillis(void)
{
    return HAL_GetTick();
}
void turnOnLED(void)
{
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
}
void turnOffLED(void)
{
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
}

```



```
}  
bool ledIsOff(void)  
{  
    return HAL_GPIO_ReadPin(LED_GPIO_Port, LED_Pin) == GPIO_PIN_RESET;  
}
```

Your application code (that second listing), is now hardware agnostic! Furthermore, you have one source file ("STM32.c") that provides implementations for the functions used by your application code that, if compiled with the right libraries and settings, will result in a binary file that you can program onto your STM32 microcontroller. Our job, now, is to find *other* implementations for those functions that fulfill the same expectations *and* that can run on your computer. (We'll focus on retargeting the IO functions in this article; I have faith in your ability to replace HAL_GetTick() with an equivalent function from `time.h` or `chrono`.)

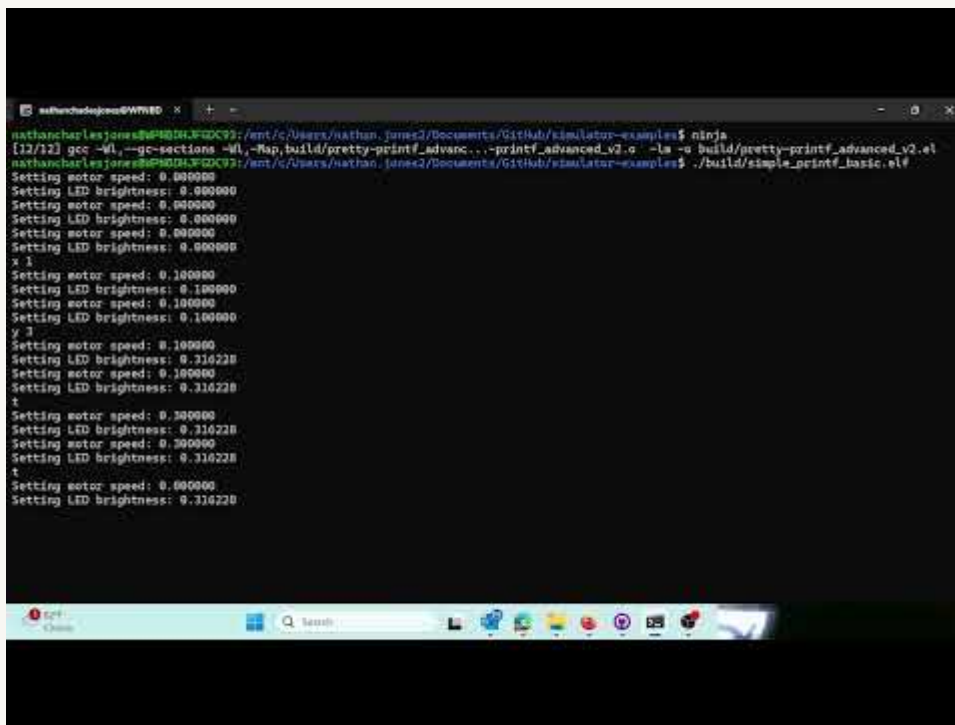
The simplest option: printf/scanf

Yup, you read that right: to create the simplest simulator we're going to implement *all* outputs with `printf` and *all* inputs with `scanf` (or a close cousin, such as `fgets` or `getchar`)! You'll compile this simulator for your laptop or desktop computer after creating a file (i.e. "x86.c") that implements any hardware-dependent functions (e.g. `display()`, `readButton()`, etc) using just `printf` or `scanf`. For example:

- Are you reading a button? Use `if(getchar() == 'a'){ ... }` (here, the 'a' key is used to simulate the button on our embedded system)
- Are you reading from a potentiometer? Use `scanf("%d", pot_val);`
- Are you sending out a PWM to a motor or LED? Use `printf("DC for motor is now %d%%\n", dc_val_for_motor_pwm);`

If you need to simulate multiple inputs, then you can treat `scanf/getchar` like a command-line interface, reading characters until a newline is reached and then parsing the string that was entered.

Here's an example of what this might look like for a simple embedded system that uses an accelerometer to control the speed of a motor and the brightness of an LED (code [here](#)). Users can interact with and test this system by entering "x", "y", or "z" followed by a double-precision floating point number to simulate the accelerometer reading those values in those directions. Typing "t" simulates double-tapping the accelerometer, which changes the direction that is used to set the motor speed (x, y, z, or total); the LED brightness is always set by the total acceleration (the sum of squares of the three acceleration values).



```
nathancharlesjones@MPC9004:~/Documents/GitHub/simulator-examples$ ninja
[12/12] gcc -O1,--gc-sections -O1,-fap,build/pretty-printf_advanc...-printf_advanced_v2.o -la -s build/pretty-printf_advanced_v2.o
nathancharlesjones@MPC9004:~/Documents/GitHub/simulator-examples$ ./build/sample_printf_basic.elf
Setting motor speed: 0.000000
Setting LED brightness: 0.000000
Setting motor speed: 0.000000
Setting LED brightness: 0.000000
Setting motor speed: 0.000000
Setting LED brightness: 0.000000
x 1
Setting motor speed: 0.100000
Setting LED brightness: 0.100000
Setting motor speed: 0.100000
Setting LED brightness: 0.100000
y 2
Setting motor speed: 0.100000
Setting LED brightness: 0.216228
Setting motor speed: 0.100000
Setting LED brightness: 0.216228
t
Setting motor speed: 0.200000
Setting LED brightness: 0.216228
Setting motor speed: 0.200000
Setting LED brightness: 0.216228
t
Setting motor speed: 0.000000
Setting LED brightness: 0.216228
```

<https://www.youtube.com/watch?v=nITBdMFNMJs>

Beware of the following "gotchas", though:

- `scanf` *won't* consume the newline (`\n`) in stdin that signals `scanf` to return; it will thus be the first character read by `scanf` on your code's next pass. If your code can't handle this rogue newline in stdin, then you'll need to clear stdin using the snippet of code discussed [here](#) after processing any input (or use `getchar()` instead).
- `printf` only *actually* prints everything you send it if it sees a newline (which is like a signal that tells `printf` to clear its buffer to stdout). At least on my machine, it seems like `printf`'s buffer is two characters long, which means that the last two characters of the most recent call to `printf` won't show up on the terminal unless I follow up that call to `printf` with a call to `fflush(stdout)` (clearing that two character buffer).

With simplicity comes trade-offs in other areas, though. Let discuss the limitations of this method and a few ways to mitigate them.

Making `scanf/getchar` non-blocking

Both `scanf` and `getchar` are blocking functions, holding a program hostage until the user types something and then presses the Enter key. Adding a call to either one anywhere in your main loop will stall the rest of your tasks, possibly causing it to fail! One solution to this is to put the call to `scanf/getchar` in its own thread, allowing the rest of your application to continue running while that thread waits for user input.

```
*****
*           x86.c           *
*****
#include <pthread.h>
```

```

#include <semaphore.h>
void* inputThread(void*);
pthread_t h_inputThread;
sem_t buttonSemaphore;
void initHardware(void)
{
    pthread_create(&h_inputThread, NULL, inputThread, NULL);
}
void* inputThread(void* data)
{
    while(1)
    {
        char c = getchar();
        if('b' == c) sem_post(&buttonSemaphore);
    }
}

```

Here, the pthread library is used to put the call to `getchar()` into its own thread. This thread can then signal to the rest of the program when a new value has been entered by any means of inter-task communication, such as setting a global variable, posting a semaphore, adding a value to a queue, etc. The code above will post to `buttonSemaphore` when the user presses 'b' + Enter. Code that is waiting for the button can simply check if the semaphore has been posted to or not.

Another solution is to read input values from a file, as opposed to typing them in by hand. We can accomplish this by creating a text file with the appropriate inputs and replacing a call to `scanf` with a call to `fread` or `fgetc`. Let's say we were building a simple embedded system that did one thing when a button was pressed:

```

*****
*          main.c          *
*****
void main(void)
{
    initHardware();
    while(1)
    {
        if(buttonIsPressed()) doStuff();
    }
}

```

and, also, that we had a text file called "input.txt" in which was stored the following sequence:

```

b----b-----b-b

```

By creating an "x86.c" file that looked like this:

```

*****
*           x86.c           *
*****
#include <stdio.h>
FILE * inputs;
void initHardware(void)
{
    inputs = fopen("inputs.txt", "r");
}
bool buttonIsPressed(void)
{
    return (fgetc(inputs) == 'b');
}

```

we can "spoof" the application into thinking that the button was pressed on the 1st, 6th, 15th and 17th times through the main loop. A more complete implementation might restart the file over once the end of the file is reached (allowing for the inputs to repeated over and over again). A more advanced implementation (left to the reader) might:

- use the text file to specify *when* the output of `buttonIsPressed` should be true (e.g. the contents of the text file become 1,6,15,17) to avoid typing hundreds of dashes, or
- put `buttonIsPressed` inside a task that only runs at 10 or 100 Hz so that the timing of the user inputs is better known, ex:

```

*****
*           x86.c           *
*****
#include <stdio.h>
FILE * inputs;
void initHardware(void)
{
    inputs = fopen("inputs.txt", "r");
}
bool buttonIsPressed(void)
{
    static unsigned long next = -1;
    static unsigned long period = 10; // Run task at 100 Hz (i.e. every 10 ms)
    if(next - getMillis() > period)
    {
        next += period;
        return (fgetc(inputs) == 'b');
    }
    else return false;
}

```

By a similar token, we can replace (or add to) `printf` with `fwrite`. If we write data to a file in a common format like CSV, XML, or JSON, we have a detailed log of every output our microcontroller changed during the course of its execution! From a testing perspective, we can use this to confirm that the application behaves the way we expect (e.g. "after input X is applied at 100 ms after program initiation, our program should set the motor speed to Y"). Or we could simply feed this data to another program (like Excel) to visualize what happened on our microcontroller after each execution.

A Note About Race Conditions

(UPDATE**: Added 07 Oct 2024)** However, now that our program has two independent threads of execution (our application code and the thread running `scanf`), we have to be cautious to not introduce any race conditions into our code. This could accidentally happen in a handful of different ways.

1. **`scanf` writes to a variable non-atomically that another thread reads from or writes to at any point.**

This is a data race and could result in erroneous values if that thread reads from or writes to the shared variable in the middle of `scanf` also writing to it (or vice versa, depending on which thread interrupts which). Variables which can be accessed atomically on your system are probably safe, though its not guaranteed; it's possible that `scanf` uses an *atomic* variable *non-atomically* while it's writing to it and, if it did, then we'd still have a race condition. For example, maybe `scanf` parses an integer one digit at a time, updating the variable live as a user types "1" (setting the variable to 1) then "4" (updating the variable to 14) then "5" (updating the variable to 145). If the application code tried to read the variable before `scanf` was done, it would get a garbage value. This would all depend on the implementation of `scanf`, of course, of which I'm ignorant.

2. **`scanf` writes to two or more variables at the same time that should all be updated together.**

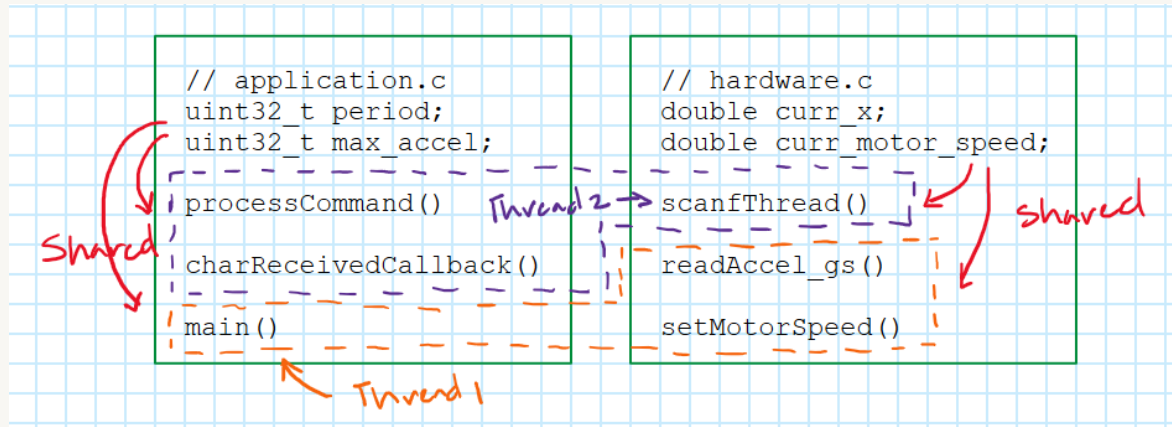
Although we wouldn't have any data races in this case, we'd still have a similar problem: data that should be accessed atomically isn't. For example, imagine we were using `scanf` to read x, y, and z acceleration values while another thread was accessing them. Without synchronization, our application code may read an old value of x but new values for y and z (or vice versa, depending on which thread is interrupting which).

3. **`scanf` is called by two or more threads.**

This is mostly just bad practice. From what I can tell, `scanf` *is* thread-safe but this *only* means that whichever `scanf` gets called first is the one that will process your input. It would be better to treat `stdin` like an MCU peripheral: use a mutex to prevent two or more threads from simultaneous access or move all accesses to `stdin` into a single module or thread. In [this example](#) ("Simple printf, Basic"), I simulate an embedded system that has a serial interface, but I also use `stdin` to get commands from the user to control the simulation. My solution was to only get user input inside a single thread and either operate

on that data (if it was intended to control the simulation) or pass it through to the application code (if it was intended for the serial link on the embedded system).

The weirdest part about this is that you might have functions in the same source file that execute in different threads. In a variation on that last example ("[Pretty printf, Advanced](#)"), the main application calls hardware-dependent functions like `readAccel_gs` and `setMotorSpeed` that read from or write to variables in the hardware-dependent source file like `curr_x` or `curr_motor_speed`. At the same time, I have a thread in the hardware-dependent source file using `getchar` to collect user input which sometimes calls public functions in the application code to set or modify variables in that source file like `max_accel` and `period`.



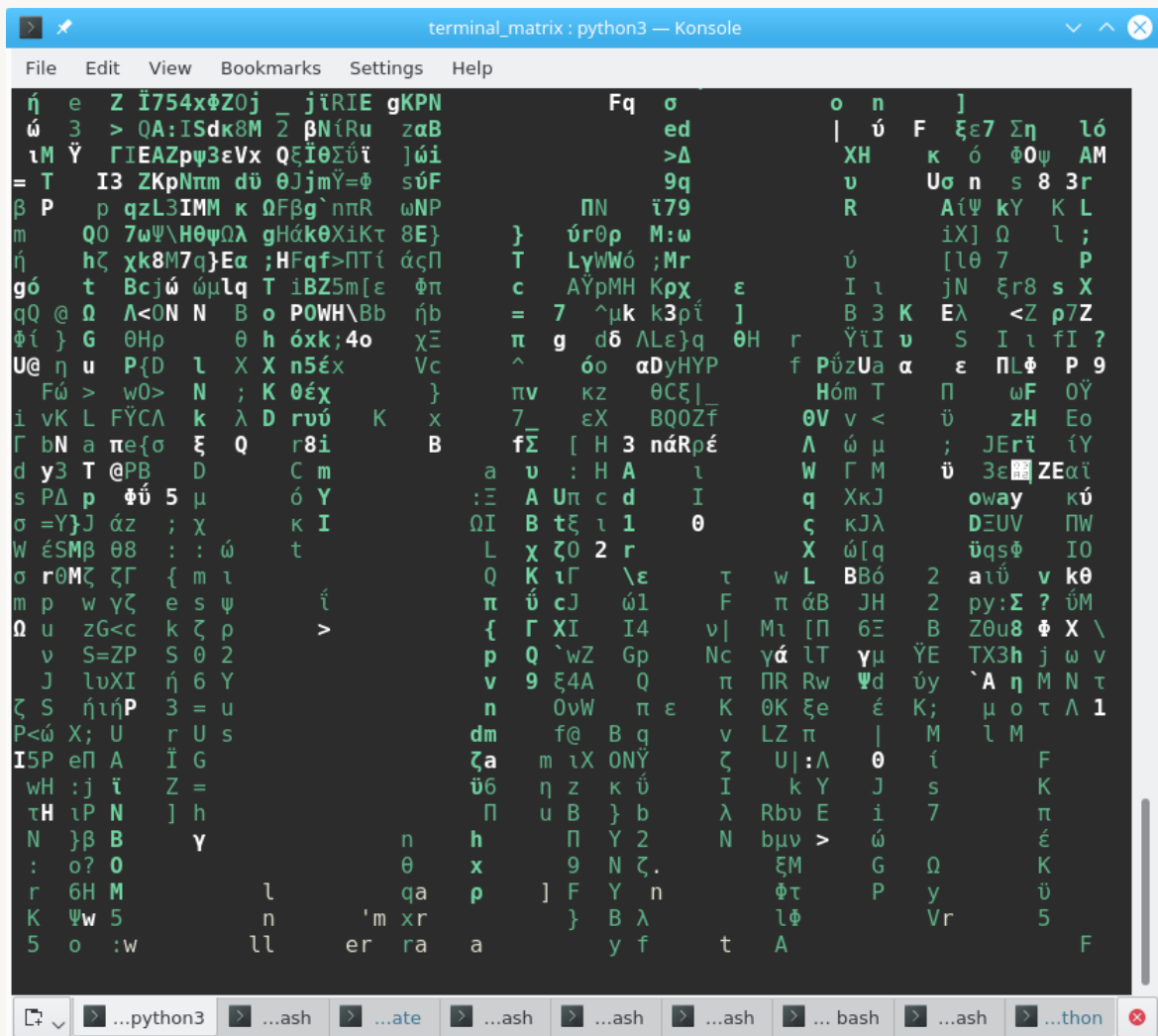
I luck out in that each of these variables can be accessed atomically, but it's important to identify those possible race conditions and ensure that none exist for your program to work correctly.

The solutions to these (and other) race conditions are to introduce some form of synchronization that will prevent simultaneous access to the data from multiple threads. For example:

1. Protect the data with a mutex or semaphore (code can't access the data, to read or write, if they can't acquire the mutex/get the binary semaphore). Or, if there is just one writer and one reader, a simpler option is to set a global flag when the data has been written to and clear the flag when its been read; that's the solution I took in a [different example](#) about trying to read from/write to three acceleration values at the same time.
2. When writing new data, post it by value to a mailbox/queue. Then any reader gets exclusive access to that piece of data by virtue of having exclusive access to its own queue. (Things get a little weirder if you want to pass a piece of data by reference or if you need to support multiple threads sending and receiving data, but that's a topic for another article.)

Making printf prettier

The output from `printf` could be less-than-useful if you're using it many dozens or hundreds of times a second and the text is flying past in your terminal so quickly you feel like you're trying to read the Matrix.



The first solution for this is to simply print out your values more slowly. We can use the same technique as above when we read from "inputs.txt" at 100 Hz:

```
void setMotorDC(int dc)
{
    static unsigned long next = -1;
    static unsigned long period = 1000; // Run task at 1 Hz (i.e. every 1000 ms)
    if(next - getMillis() > period)
    {
        printf("DC for motor is now %d%%\n", dc);
        next += period;
    }
}
```

This is how the example in the video above is configured. We could even get fancy and report things like the maximum, minimum, or average values for the motor's duty cycle.

Another useful technique is to clear the terminal screen periodically. This works particularly well if you print out the same things in the same order or if you're just printing messages that can overwrite each other. Clearing the screen will make sure the cursor starts from the same location, helping ensure that your values print out in the same place every time.

One way to clear the screen is to use `printf("\033[2J")`. This is actually a VT100 command code that was used in Medieval Europe to help control remote terminals. Many (but not all) terminal emulators, like bash, still respond to these codes, allowing you to do some interesting things with `printf`, such as:

- Change the font/foreground color of your text to bright green (RGB value of [35, 194, 40]) and the background color to bright red (RGB value of [255, 0, 0]) (from [this](#) article):

```
printf("\033[38;2;35;194;40m\033[48;2;255;0;0m");
```

```
nathancharlesjones@WPNBHJFGDC93:/mnt/c/Users/nathan.jones2/Documents/vt100-test$ ./main.out
HELLO WORLD
```

- Print flashing words by moving the cursor back a line (example code [here](#))

```
nathancharlesjones@WPNBHJFGDC93:/mnt/c/Users/nathan.jones2/Documents/vt100-test$ ./main.out
HELLO
```

```
nathancharlesjones@WPNBHJFGDC93:/mnt/c/Users/nathan.jones2/Documents/vt100-test$ ./main.out
WORLD
```

The most helpful ones, IMHO, are these:

- `"\033[2J"`: (previously mentioned) Clears the terminal (leaving the cursor where it is)
- `"\033c"`: Resets the terminal (cursor returns to the top-left corner)
- `"\033[<y>;<x>H"`: Send the cursor to position (<y>, <x>), where the top-left corner is (1,1) (Notice the **y-coordinate comes first!**).
 - The <y> and <x> are technically optional and default to 1 when unspecified;
 - `"\033[;H"` also sends the cursor to the top-left corner of the terminal.
- `"\0337"`: Save the state of the cursor
- `"\0338"`: Restore the state of the cursor

Given their age, these codes aren't exactly guaranteed to work with all terminals, so YMMV.

Lastly, we can also get fancy with just our ASCII characters! If we want to print out the same values in the same places, we can make little ASCII drawings to put boxes around those values or to draw arrows from one box to another.

```
/* Created using asciiflow.com */
char background[] = "+-----+          +-----+\n"
                  "| Dashboard | Pot | 758 | \n"
                  "+-----+      val | (2.440V) | \n"
                  "      +-----+      +-----+\n"
                  " Motor|101 RPM| Button status \n"
                  " speed|          | +-----+\n"
```



```
"      +-----+ |Not Pressed | \n"
"                                +-----+ \n";
```

We can incorporate these box drawings into our code in two ways.

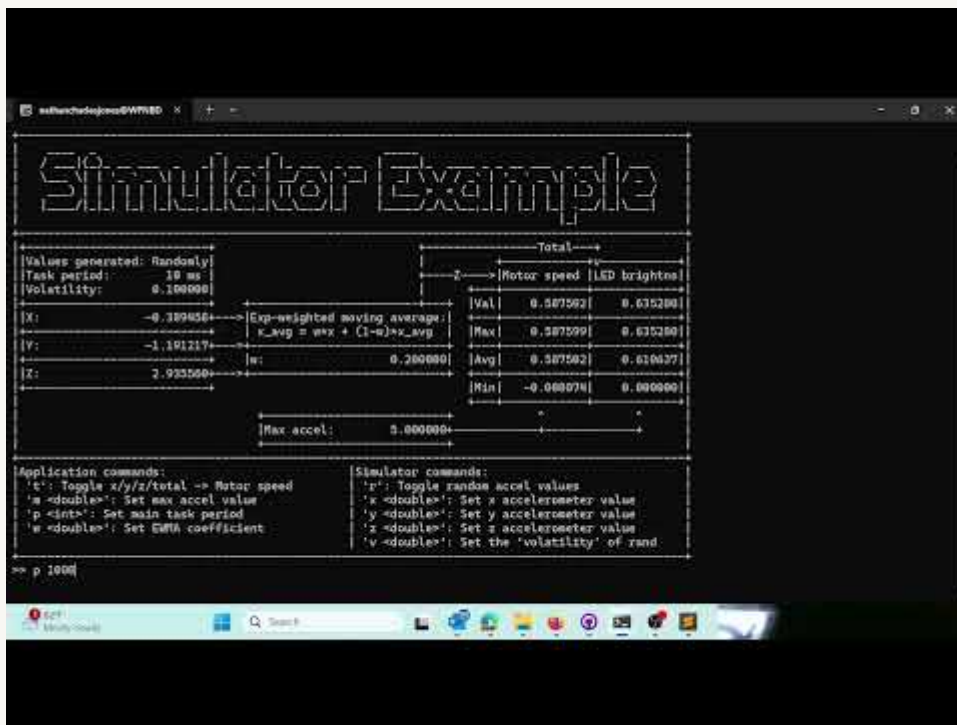
First, you can replace the spaces where your values will go with the right [printf formatting specification](#) (e.g. %-8s prints a minimum of 8 characters from a string, left-justified; %05d prints at least 5 digits of an integer padded with 0s in front, etc). The template might then look like this:

```
/* Created using asciiflow.com */
char background[] = "+-----+          +-----+ \n"
    "| Dashboard | Pot | %.4d | \n"
    "+-----+      val | (%1.3fV) | \n"
    "      +-----+          +-----+ \n"
    " Motor|%.4d RPM| Button status \n"
    " speed|          | +-----+ \n"
    "      +-----+  |%-11.11s" " | \n"
    "                                +-----+ \n";
```

Then periodically execute `printf(background, pot_val, pot_Volt, motor_speed, button_status);` to update the display. If your values don't have convenient "getter" functions, then they can be stored or copied into local variables for this call to `printf` to access.

(What's going on with this string definition? C will treat adjacent strings as being contiguous, so `char name[] = "Na" "than"` is the same as `char name[] = "Nathan"` in C. That's how each line gets "concatenated" with the one before. This also helps us preserve our layout, even after we include all of the format strings. For example, the format string `%-11.11s` will take up 11 characters when it's printed to the terminal but only 8 characters when it shows up in our string definition. To preserve the ASCII layout of our "background" in our code, we'll close off the string and add enough space so that the whole thing takes up exactly as much space as it will when it's printed. Now, `%-11.11s" "`, with those two extra quotation marks and the space in the middle that the compiler will ignore when it puts the string before and after it together, takes up exactly 11 characters on our screen, just like the string will when its printed. See [here](#) for information about the `%-11.11s` format string.)

Here's an example of what it might look like if we went wild with this technique! Using [ASCIIFlow](#) and [this website](#), I created the diagram you can see in the video below, with [spots in the diagram for each of the variables](#) I wanted to be printed. The terminal is [cleared and re-printed](#) anytime a variable gets updated, which is fast enough that the display feels like its updating in near-real time. A [pthread](#) reads input from the user (using `getchar()`) and either uses it to change aspects of the simulator (if the command is 'r', 'x', 'y', 'z', or 'v') or passes it to the application code to change the state of the application code (if the command is 't', 'm', 'p', or 'w').



<https://www.youtube.com/watch?v=erZ2x05wkJA>

Second, you can leave the template as-is and use VT100 codes to put the cursor in the right spot when a new value is "outputted". For example, if we want to post a new motor speed we could put the cursor at position (7,4) (assuming the top-left corner of the "Dashboard" box is at [1,1]), and use `printf("%3d", motor_speed)` to put three integer digits right before the letters "RPM". This will mean that you have fewer overall screen updates, but the downside is that you'll need to manage the individual screen coordinates for every item you want printed.

What can I do with printf/scanf?

As cool as that last demonstration was (and it *was* cool, amiright??), maybe you're still a little underwhelmed at the thought of using `printf` and `getchar` to simulate your embedded system. I mean, it feels so *different* from the real thing, doesn't it? How could it be useful?

The best answer I can give is that, after "isolating the madness", your application code *doesn't care how those functions are implemented*. In other words, `printf` and `getchar` are every bit as real to your code as your GPIO or I2C functions are, even if they don't feel real to you. Even with just `printf` and `getchar`, you can fully test your application code to ensure that it works well ahead of any hardware being ready to test it on. Trust me, I found so many bugs in the [code that I was developing for this article](#) *exactly* because I was able to run it and test it and see it work (or not work)!

Furthermore, my simulations don't cease to serve a purpose even after I get my hands on some hardware.

- When I'm ready to port my code to my microcontroller, I can replace `printf/getchar` with `UART_write/_read` and use my "simulator" as an interface to my MCU. This would help me prove that my code executes as expected after being compiled for a brand-new target.

- Then, I could *add* (not replace) my outputs and inputs one at a time. By keeping around my UART_write/_read functions while I add my GPIO and I2C functions, I can watch my MCU as its operating and even control it from my computer.
- Later still, after I'm finished bringing up my embedded project, my "simulator" can make an excellent dashboard for my system, showing me a live picture of what's going on in my microcontroller. (If I swap out my UART functions for BLE or WiFi functions, I could even control my MCU wirelessly!)

Summary

Simulating your embedded project on your computer is a good thing; so good that Uri Shaked calls it a "superpower". With it, you have an unprecedented ability to test and execute your code, helping you not only develop faster and with fewer errors, but also giving you an excellent tool for interacting with your MCU throughout your development.

The first thing to do if you want to simulate your code is to "isolate the madness": pull out any hardware-dependent code into its own source file. Keep your application code completely unaware of however those functions will be implemented.

The simplest method for simulating your code is just to replace any outputs with calls to `printf` and any inputs with calls to `scanf` or `getchar`; `scanf`/`getchar` should be put into a thread to allow the rest of your program to run while waiting for user input. (Alternatively, you could implement inputs/outputs with file reads/writes.)

The output from `printf` can be made easier on the eyes with ASCII art, the right format strings, and some VT100 codes that clear the terminal and move the cursor around.

And that's all it takes to make a very capable (and quite slick, IMHO) simulator for your embedded project! Thanks for sticking around this far; I hope you've found something useful here! In the next article, we'll look at a few other simulation techniques that look *even cooler*, though may potentially require more work to pull off.

Resources

- VT100 codes
 - <https://medium.com/israeli-tech-radar/terminal-esc...>
 - <https://www.programminglogic.com/playing-with-text...>
 - <https://espterm.github.io/docs/VT100%20escape%20co...>